# Cache Memories

CSE 2138: Systems Programming


## Instructor:

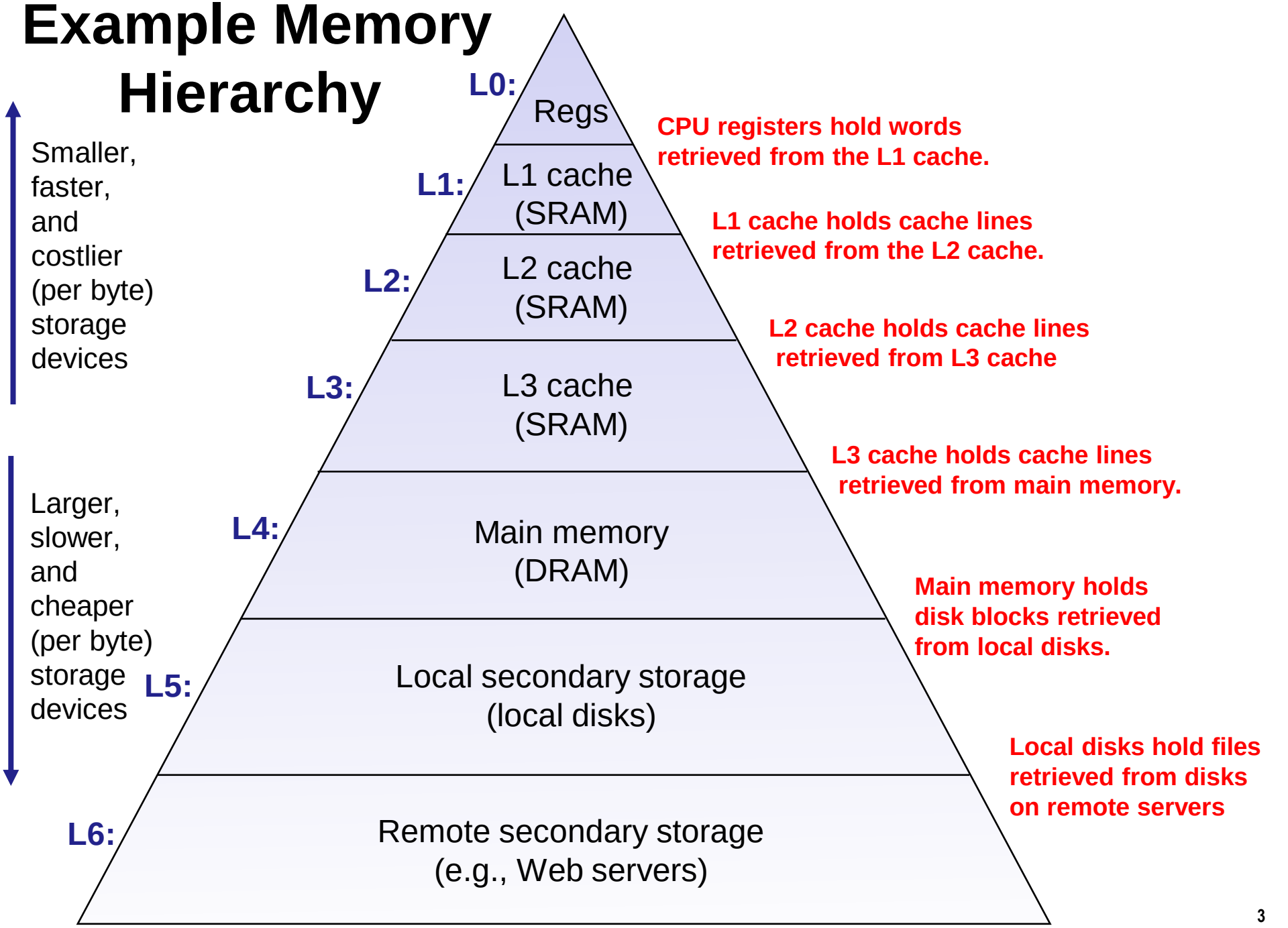
Betül BOZ


*Slides adapted from Bryant & O'Hallaron's slides*

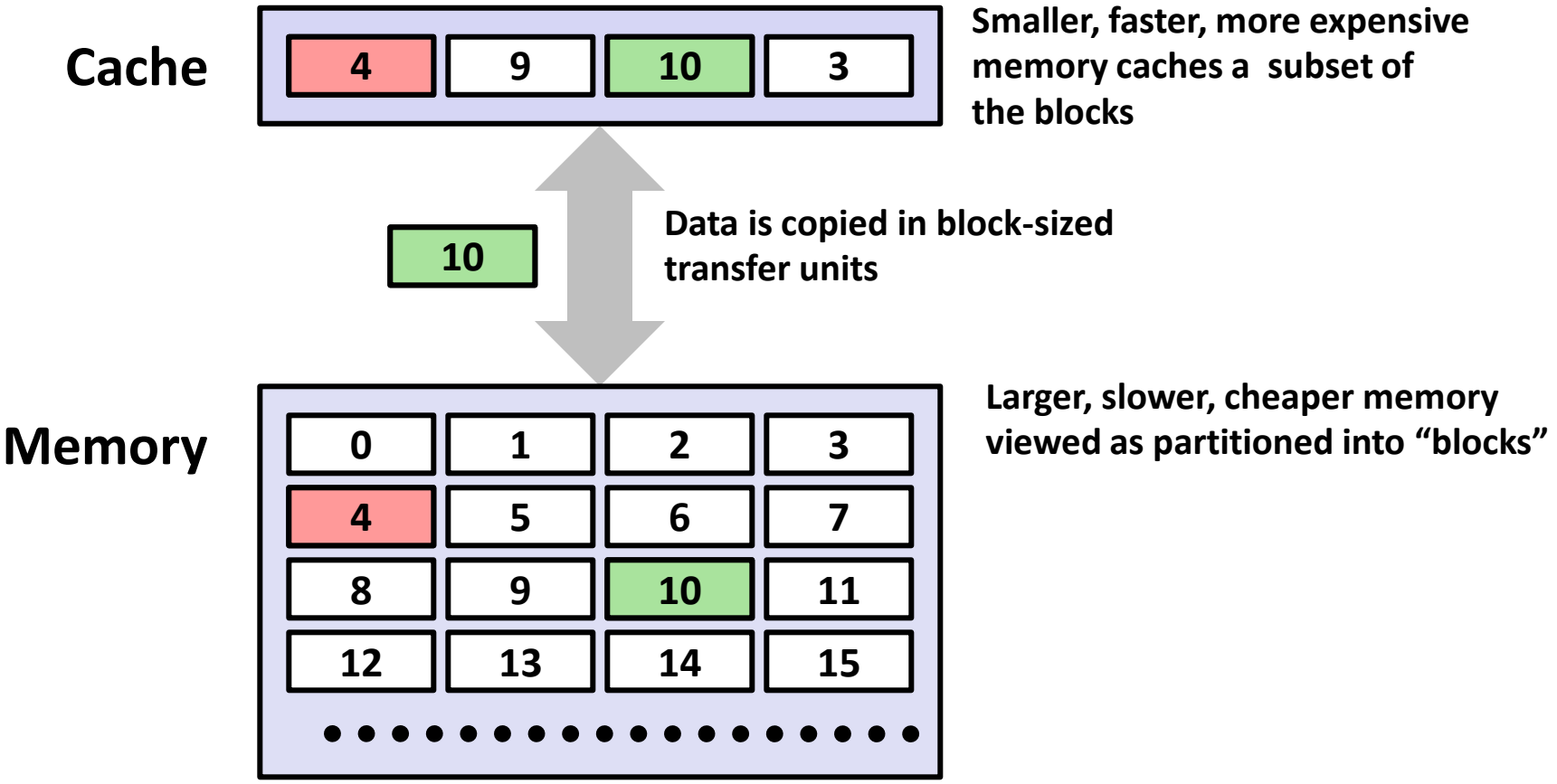
# Today

- **Cache memory organization and operation**
- **Performance impact of caches**
  - Rearranging loops to improve spatial locality

# Example Memory Hierarchy

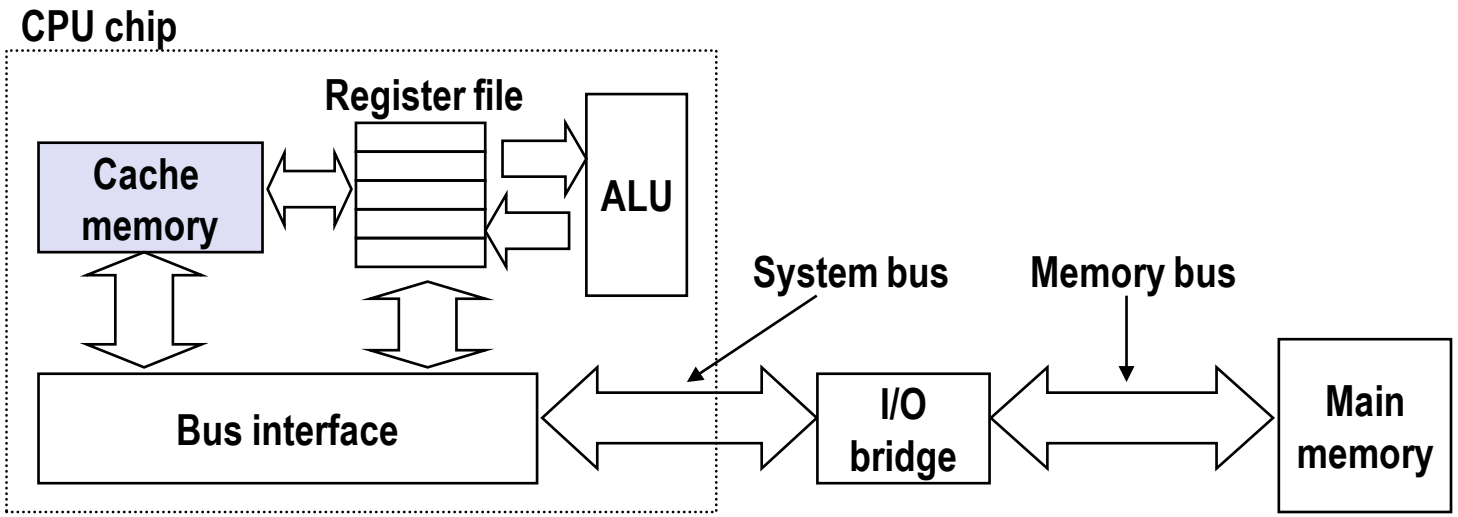


# General Cache Concept

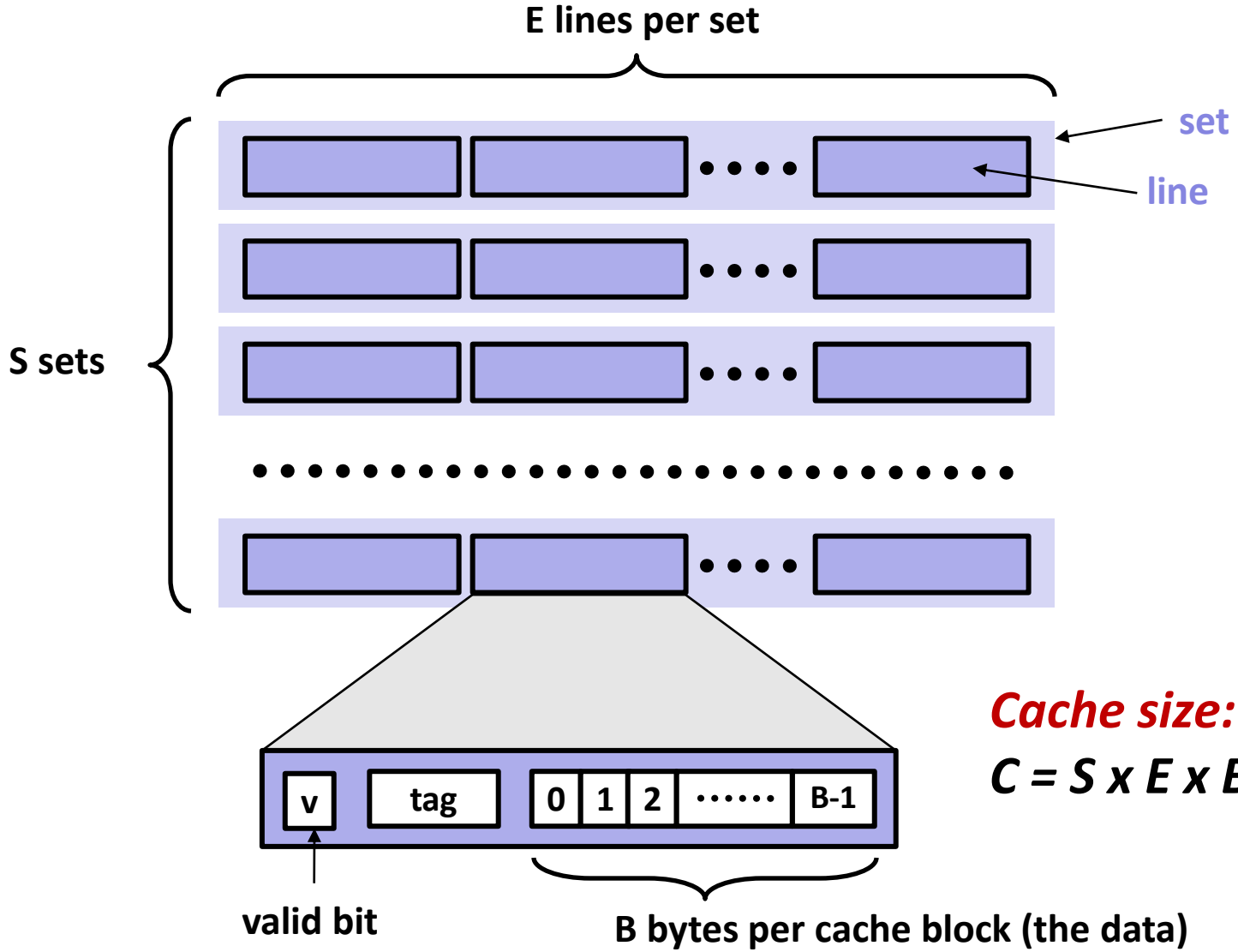


# Cache Memories

- **Cache memories are small, fast SRAM-based memories managed automatically in hardware**
  - Hold frequently accessed blocks of main memory
- **CPU looks first for data in cache**
- **Typical system structure:**



# General Cache Organization (S, E, B)

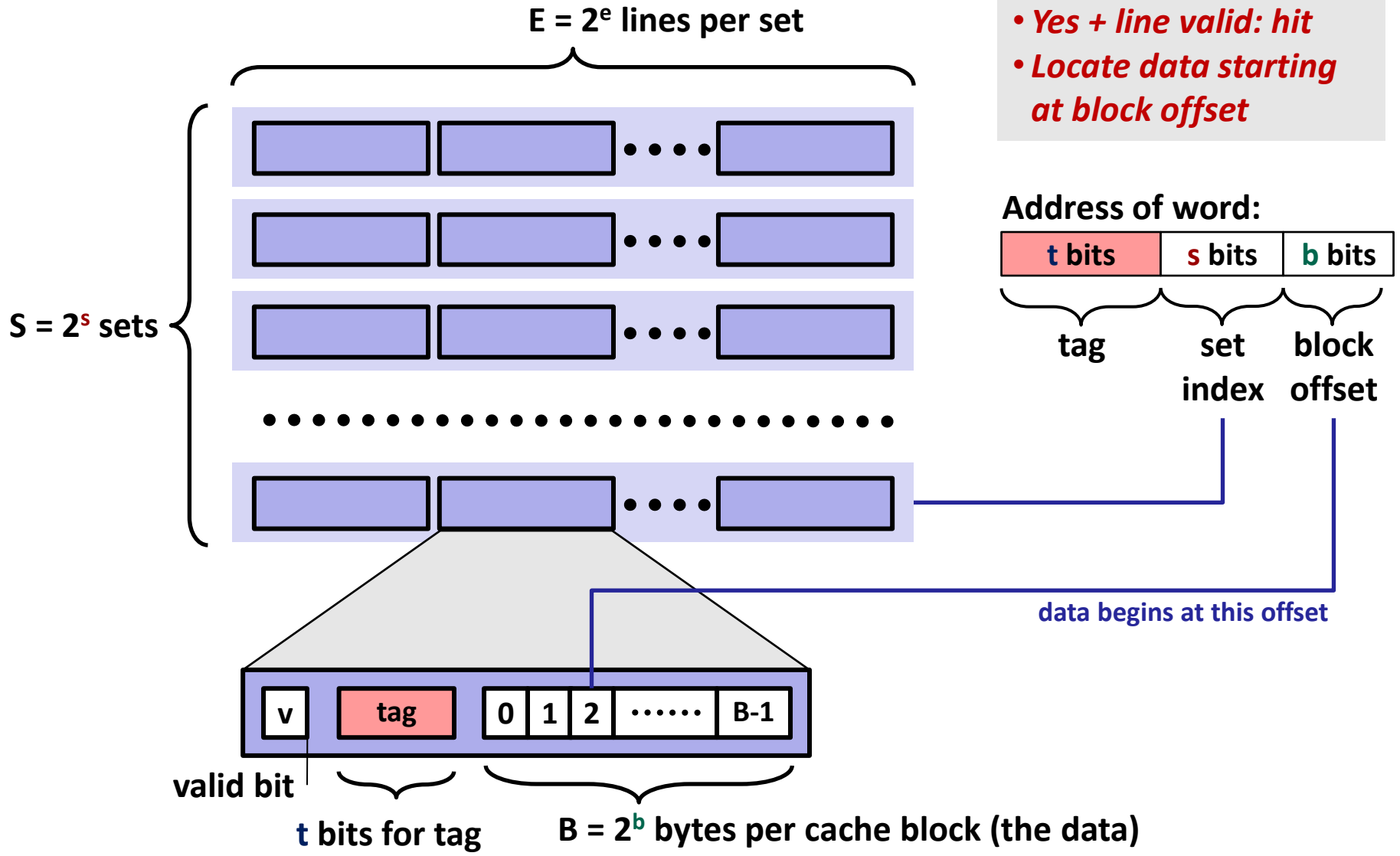



***Cache size:***

***C = S x E x B data bytes***

# Cache Read

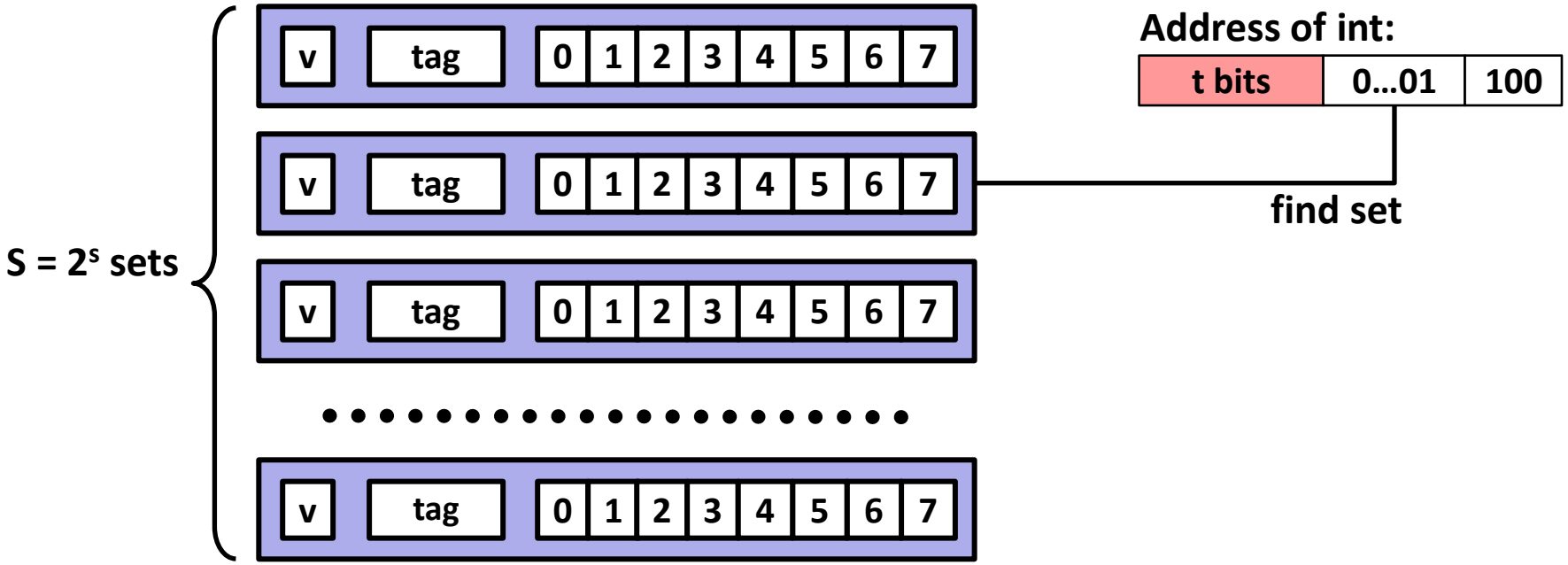
- *Locate set*
- *Check if any line in set has matching tag*
- *Yes + line valid: hit*
- *Locate data starting at block offset*



# Example: Direct Mapped Cache (E = 1)

**Direct mapped: One line per set**

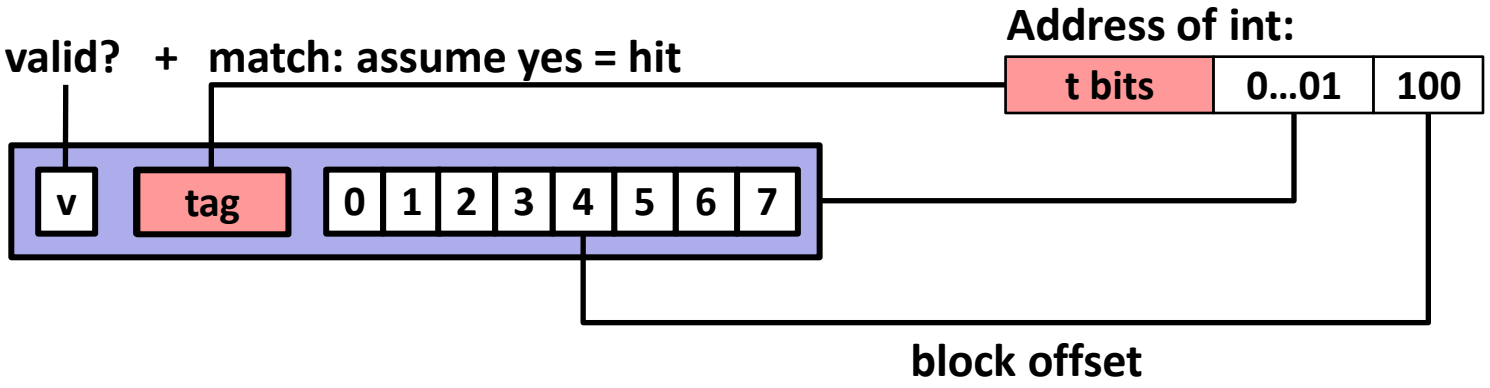
**Assume: cache block size 8 bytes**



# Example: Direct Mapped Cache (E = 1)

**Direct mapped: One line per set**

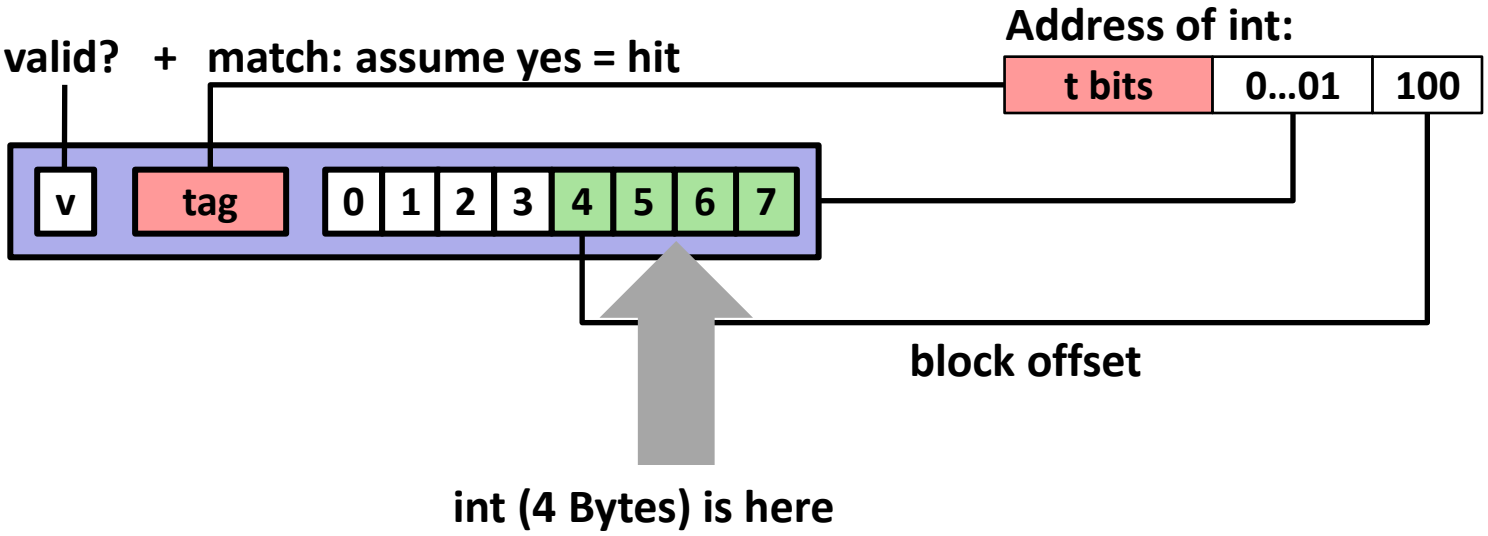
**Assume: cache block size 8 bytes**



# Example: Direct Mapped Cache (E = 1)

**Direct mapped: One line per set**

**Assume: cache block size 8 bytes**




**If tag doesn't match:** old line is evicted and replaced

# Direct-Mapped Cache Simulation

| t=1 | s=2 | b=1 |
|---|---|---|
| x | xx | x |

M=16 bytes (4-bit addresses),
B=2 bytes/block, S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

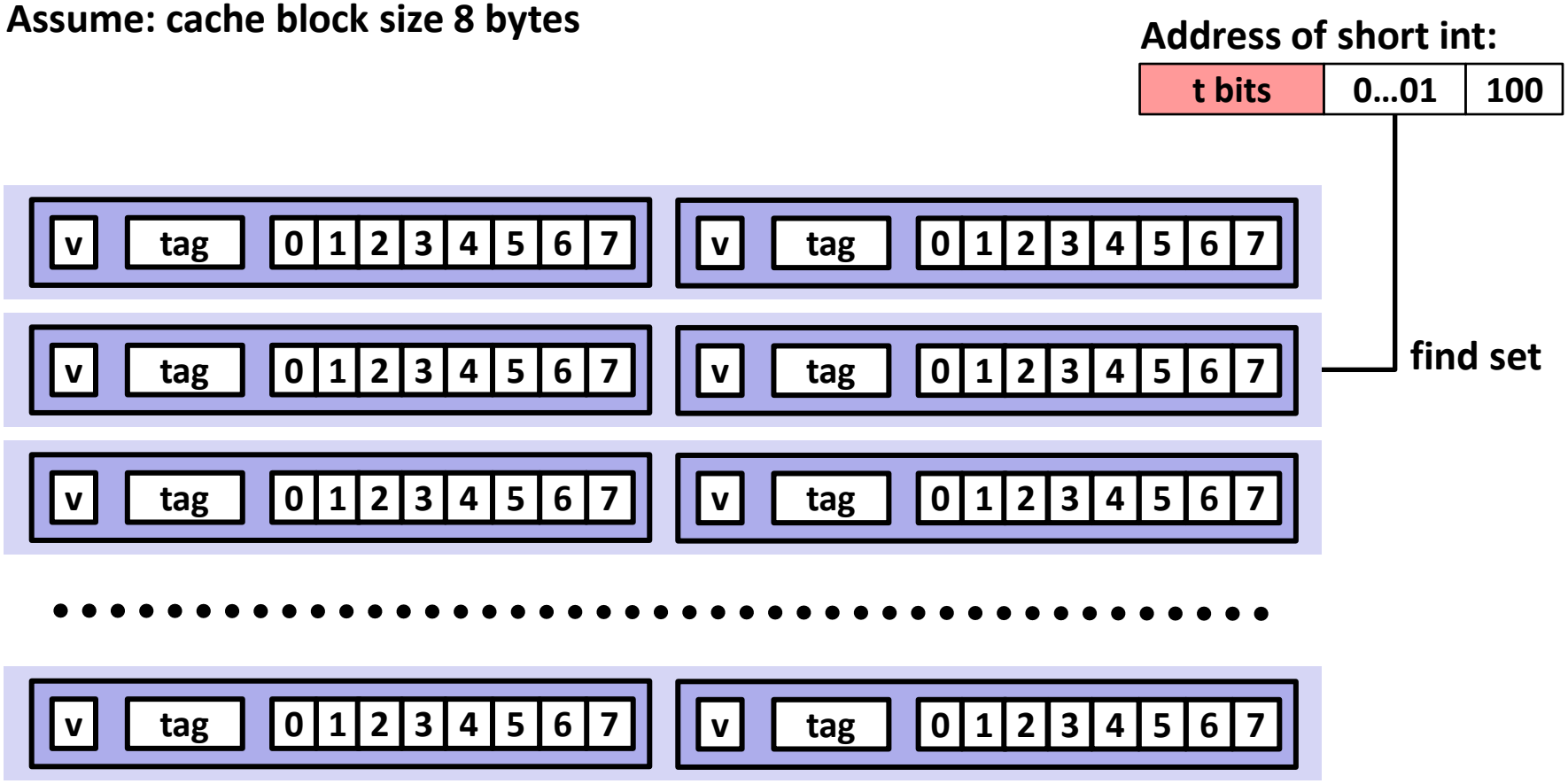
| | | |
|---|---|---|
| **0** | **[$0000_2$],** | miss |
| **1** | **[$0001_2$],** | hit |
| **7** | **[$0111_2$],** | miss |
| **8** | **[$1000_2$],** | miss |
| **0** | **[$0000_2$]** | miss |

| | v | Tag | Block |
|---|---|---|---|
| **Set 0** | 1 | 0 | M[0-1] |
| **Set 1** | | | |
| **Set 2** | | | |
| **Set 3** | 1 | 0 | M[6-7] |

# E-way Set Associative Cache (Here: E = 2)

**E = 2: Two lines per set**

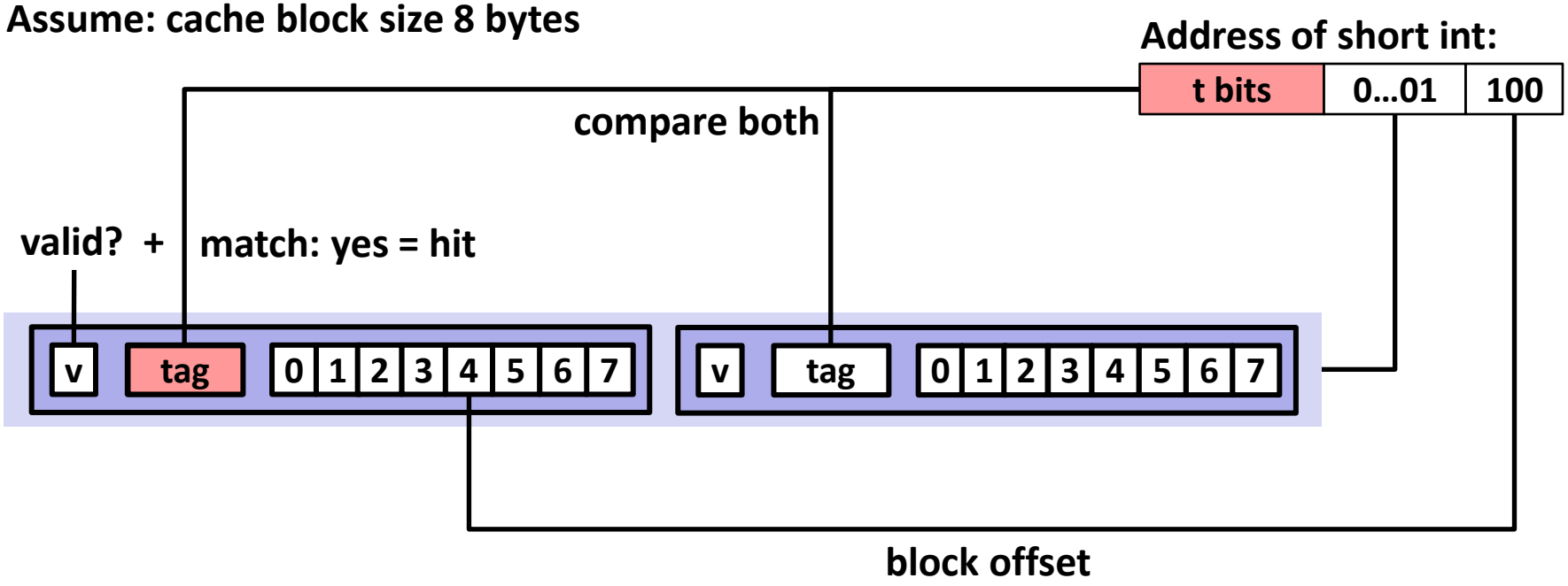
**Assume: cache block size 8 bytes**



# E-way Set Associative Cache (Here: E = 2)

**E = 2: Two lines per set**

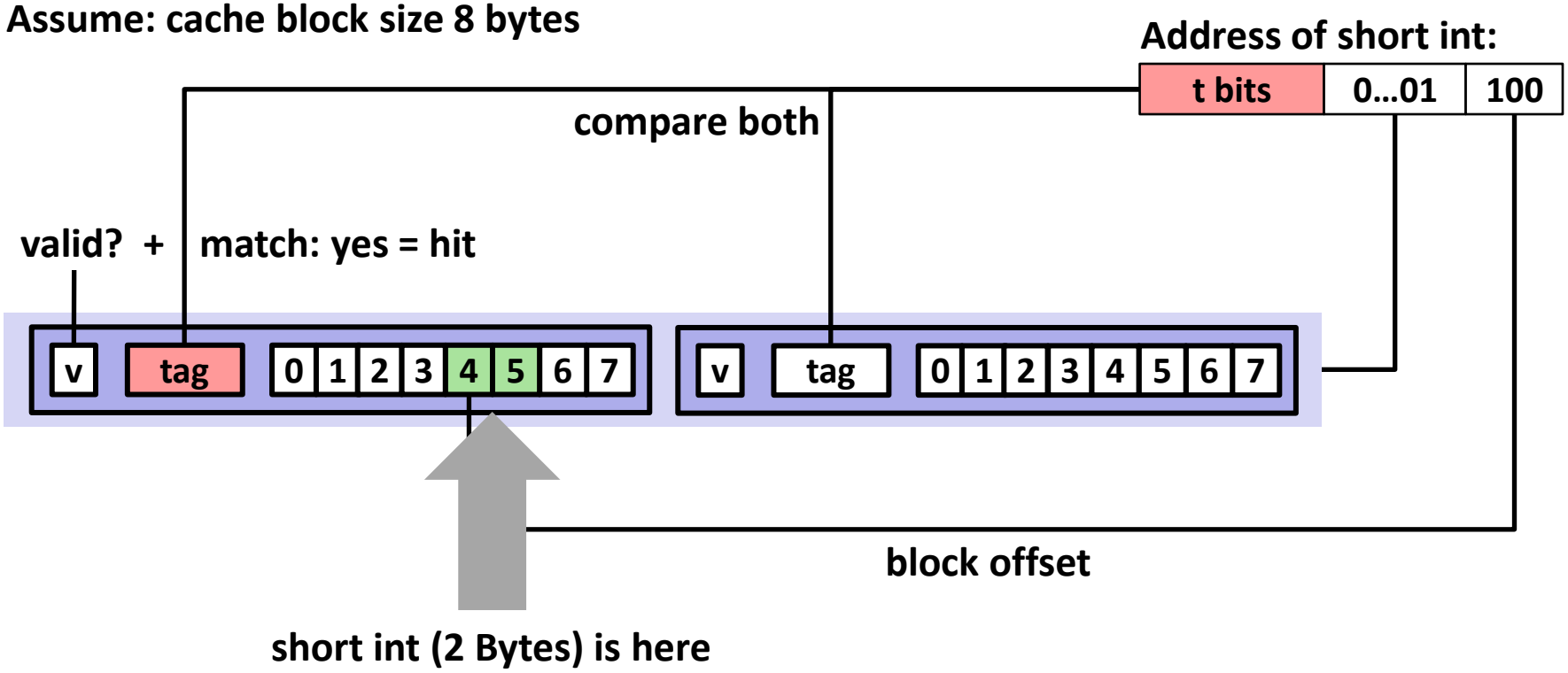
**Assume: cache block size 8 bytes**



# E-way Set Associative Cache (Here: E = 2)

**E = 2: Two lines per set**

**Assume: cache block size 8 bytes**




**No match:**

- One line in set is selected for eviction and replacement
- Replacement policies: random, least recently used (LRU), …

# 2-Way Set Associative Cache Simulation

| t=2 | s=1 | b=1 |
|---|---|---|
| xx | x | x |

M=16 byte addresses,
B=2 bytes/block, S=2 sets, E=2 blocks/set

Address trace (reads, one byte per read):

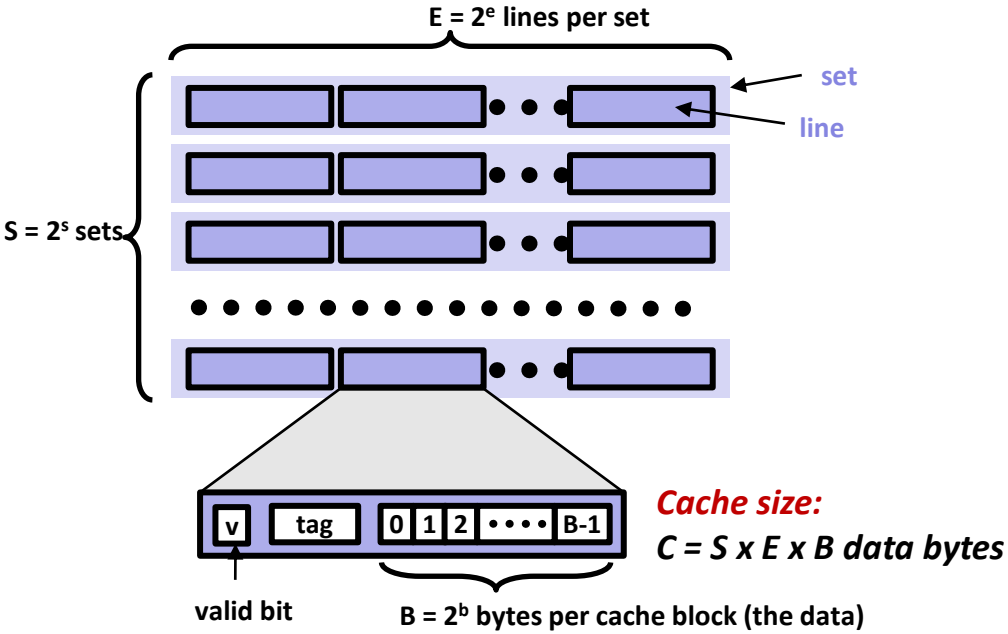
| | | |
|---|---|---|
| **0** | **[$0000_2$],** | miss |
| **1** | **[$0001_2$],** | hit |
| **7** | **[$0111_2$],** | miss |
| **8** | **[$1000_2$],** | miss |
| **0** | **[$0000_2$]** | hit |

| | v | Tag | Block |
|---|---|---|---|
| **Set 0** | 1 | 00 | M[0-1] |
| | 1 | 10 | M[8-9] |
| **Set 1** | 1 | 01 | M[6-7] |
| | 0 | | |

# Example: Core i7 L1 Data Cache

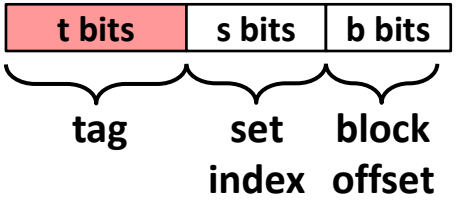
32 KB cache
8-way set associative
64 bytes/block
47 bit address range

B = ???
S = ??? , s = ???
E = ??? , e = ???
C = ???




| Hex | Decimal | Binary |
|---|---|---|
| 0 | 0 | 0000 |
| 1 | 1 | 0001 |
| 2 | 2 | 0010 |
| 3 | 3 | 0011 |
| 4 | 4 | 0100 |
| 5 | 5 | 0101 |
| 6 | 6 | 0110 |
| 7 | 7 | 0111 |
| 8 | 8 | 1000 |
| 9 | 9 | 1001 |
| A | 10 | 1010 |
| B | 11 | 1011 |
| C | 12 | 1100 |
| D | 13 | 1101 |
| E | 14 | 1110 |
| F | 15 | 1111 |

Address of word:




Block offset: ??? bits
Set index: ??? bits
Tag: ??? bits

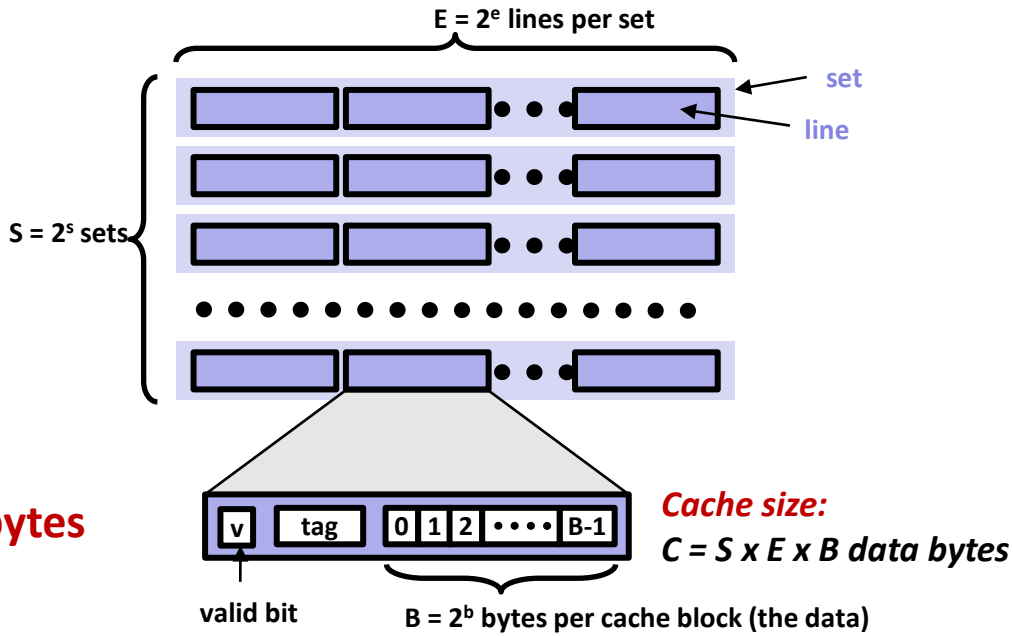
Stack address:
0x00007f7262a1e010

Block offset: 0x??
Set index: 0x??
Tag: 0x??

# Example: Core i7 L1 Data Cache

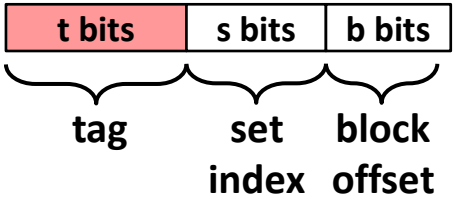
32 KB cache
8-way set associative
64 bytes/block
47 bit address range

B = 64
S = 64 , s = 6
E = 8 , e = 3
C = 64 x 64 x 8 = 32,768 bytes




| Hex | Decimal | Binary |
|---|---|---|
| 0 | 0 | 0000 |
| 1 | 1 | 0001 |
| 2 | 2 | 0010 |
| 3 | 3 | 0011 |
| 4 | 4 | 0100 |
| 5 | 5 | 0101 |
| 6 | 6 | 0110 |
| 7 | 7 | 0111 |
| 8 | 8 | 1000 |
| 9 | 9 | 1001 |
| A | 10 | 1010 |
| B | 11 | 1011 |
| C | 12 | 1100 |
| D | 13 | 1101 |
| E | 14 | 1110 |
| F | 15 | 1111 |

Address of word:




Block offset: 6 bits
Set index: 6 bits
Tag: 35 bits (47-6-6)

**Stack address:**

0x00007f7262a1e010

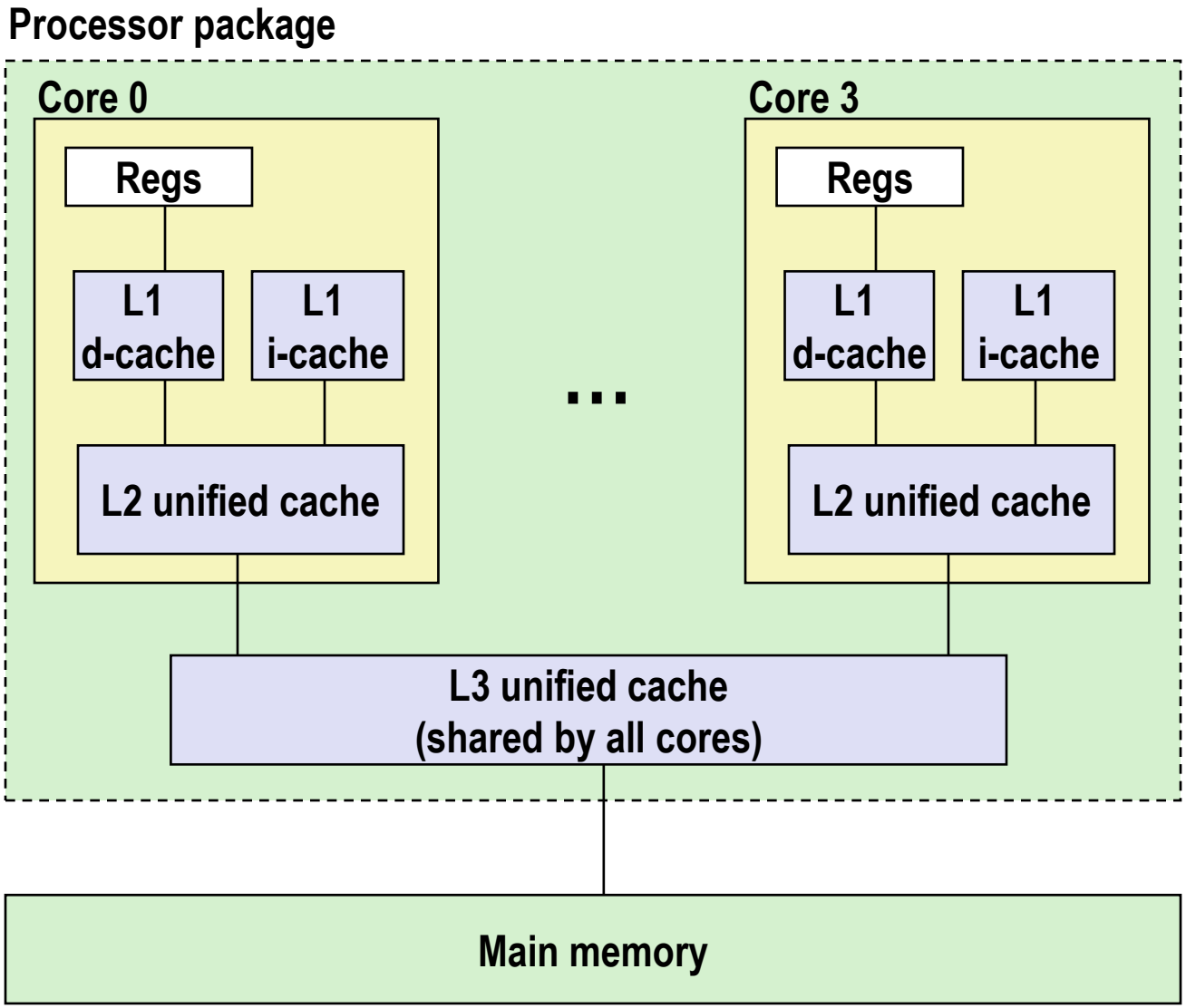
0000 0001 0000

Block offset: 0x10
Set index: 0x0
Tag: 0x00007f7262a1e

# What about writes?

- **Multiple copies of data exist:**
  - L1, L2, L3, Main Memory, Disk
- **What to do on a write-hit?**
  - Write-through (write immediately to memory)
  - Write-back (defer write to memory until replacement of line)
    - Need a dirty bit (line different from memory or not)
- **What to do on a write-miss?**
  - Write-allocate (load into cache, update line in cache)
    - Good if more writes to the location follow
  - No-write-allocate (writes straight to memory, does not load into cache)
- **Typical**
  - Write-through + No-write-allocate
  - **Write-back + Write-allocate**

# Intel Core i7 Cache Hierarchy




**L1 i-cache and d-cache:**
32 KB, 8-way,
Access: 4 cycles

**L2 unified cache:**
256 KB, 8-way,
Access: 10 cycles

**L3 unified cache:**
8 MB, 16-way,
Access: 40-75 cycles

**Block size**: 64 bytes for all caches.

# Cache Performance Metrics

- **Miss Rate**
  - Fraction of memory references not found in cache (misses / accesses) = 1 – hit rate
  - Typical numbers (in percentages):
    - 3-10% for L1
    - can be quite small (e.g., < 1%) for L2, depending on size, etc.
- **Hit Time**
  - Time to deliver a line in the cache to the processor
    - includes time to determine whether the line is in the cache
  - Typical numbers:
    - 4 clock cycles for L1
    - 10 clock cycles for L2
- **Miss Penalty**
  - Additional time required because of a miss
    - typically 50-200 cycles for main memory (Trend: increasing!)

# Let's think about those numbers

- **Huge difference between a hit and a miss**
  - Could be 100x, if just L1 and main memory

- **Would you believe 99% hits is twice as good as 97%?**
  - Consider:
    cache hit time of 1 cycle
    miss penalty of 100 cycles

  - Average access time:

    97% hits: 1 cycle + 0.03 * 100 cycles = **4 cycles**

    99% hits: 1 cycle + 0.01 * 100 cycles = **2 cycles**

- **This is why "miss rate" is used instead of "hit rate"**

# Writing Cache Friendly Code

- **Make the common case go fast**
  - Focus on the inner loops of the core functions

- **Minimize the misses in the inner loops**
  - Repeated references to variables are good (temporal locality)
  - Stride-1 reference patterns are good (spatial locality)

# Today

- Cache organization and operation
- Performance impact of caches
  - Rearranging loops to improve spatial locality

# Matrix Multiplication Example

- **Description:**
  - Multiply *n x n* matrices
  - Matrix elements are `doubles` (8 bytes)
  - $O(n^3)$ total operations
  - n reads per source element
  - n values summed per destination
    - but may be able to hold in register

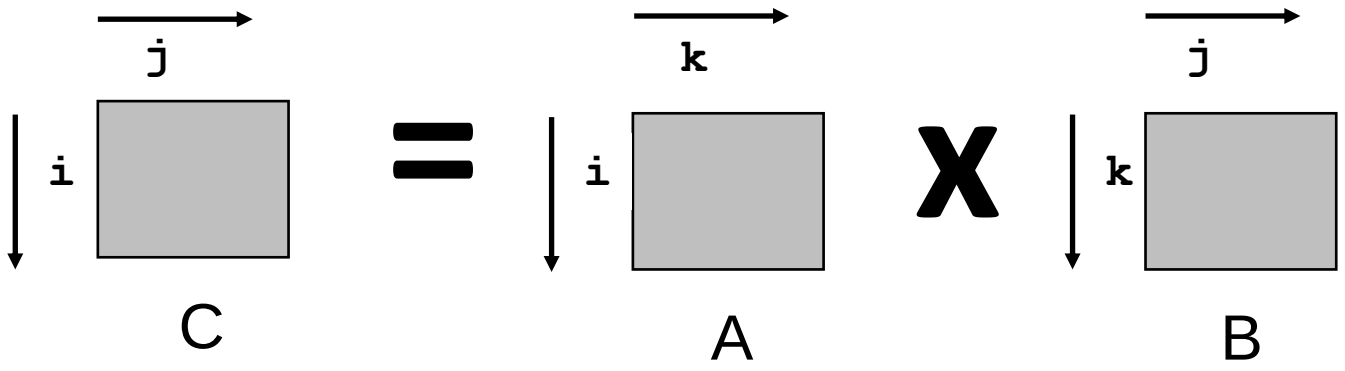
```
/* ijk */
for (i=0; i<n; i++)  {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

*Variable* ***sum*** *held in register*

*matmult/mm.c*

# Miss Rate Analysis for Matrix Multiply

- **Assume:**
  - Cache block size = 32B (big enough for four doubles)
  - Matrix dimension (n) is very large
    - Approximate 1/n as 0.0
  - Cache is not even big enough to hold more than one row
- **Analysis Method:**
  - Look at access pattern of inner loop



# Layout of C Arrays in Memory (review)

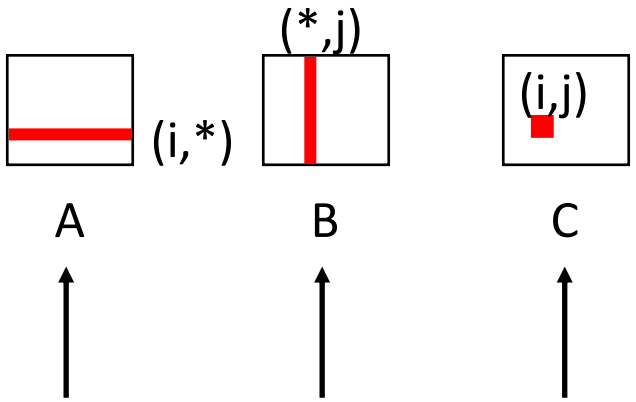
- **C arrays allocated in row-major order**
  - each row in contiguous memory locations
- **Stepping through columns in one row:**
  - `for (i = 0; i < n; i++)`
    `sum += a[0][i];`
  - accesses successive elements
  - if block size (B) > sizeof($a_{ij}$) bytes, exploit spatial locality
    - miss rate = sizeof($a_{ij}$) / B
- **Stepping through rows in one column:**
  - `for (i = 0; i < n; i++)`
    `sum += a[i][0];`
  - accesses distant elements
  - no spatial locality!
    - miss rate = 1 (i.e. 100%)

# Matrix Multiplication (ijk)

```
/* ijk */
for (i=0; i<n; i++)  {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

*matmult/mm.c*

Inner loop:




Misses per inner loop iteration:

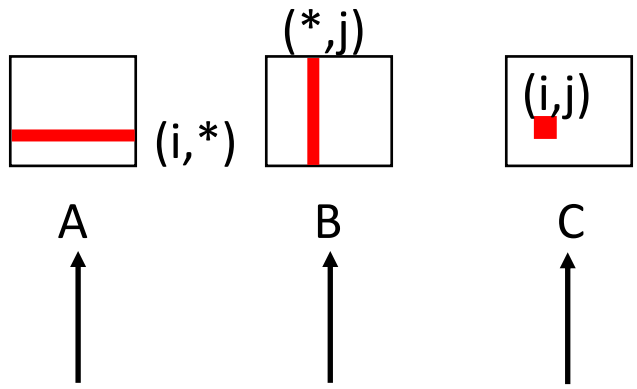
| A | B | C |
|---|---|---|
| 0.25 | 1.0 | 0.0 |

# Matrix Multiplication (jik)

```
/* jik */
for (j=0; j<n; j++) {
  for (i=0; i<n; i++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum
  }
}
```

*matmult/mm.c*

Inner loop:



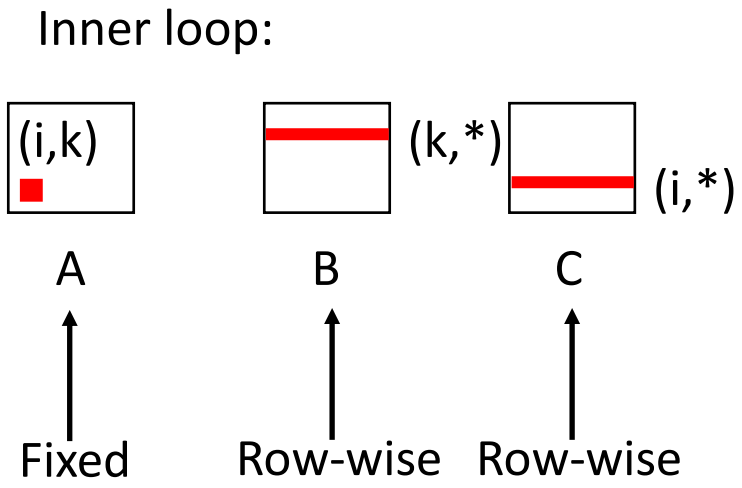

Misses per inner loop iteration:

| A | B | C |
|---|---|---|
| 0.25 | 1.0 | 0.0 |

# Matrix Multiplication (kij)

```
/* kij */
for (k=0; k<n; k++) {
  for (i=0; i<n; i++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```

matmult/mm.c



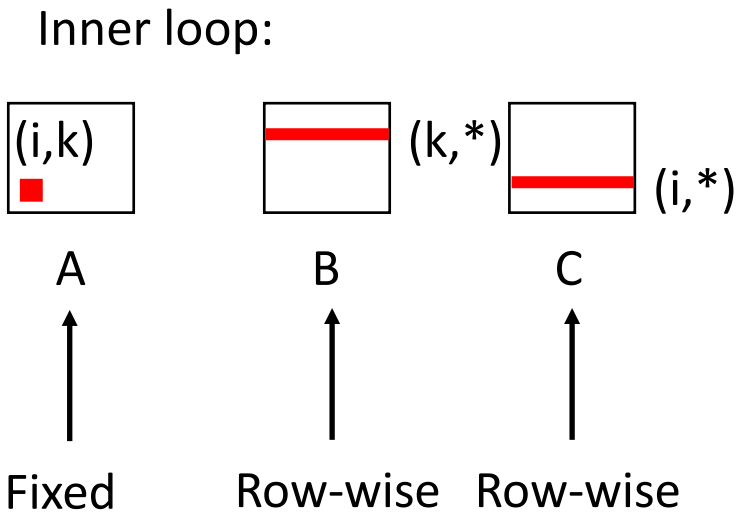

Misses per inner loop iteration:

| A | B | C |
|---|---|---|
| 0.0 | 0.25 | 0.25 |

# Matrix Multiplication (ikj)

```
/* ikj */
for (i=0; i<n; i++) {
  for (k=0; k<n; k++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```

matmult/mm.c



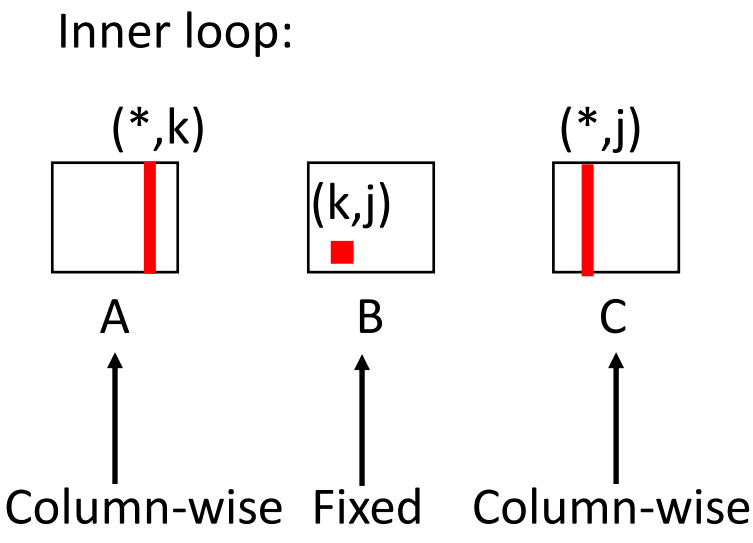

Misses per inner loop iteration:

| A | B | C |
|---|---|---|
| 0.0 | 0.25 | 0.25 |

# Matrix Multiplication (jki)

```
/* jki */
for (j=0; j<n; j++) {
  for (k=0; k<n; k++) {
    r = b[k][j];
    for (i=0; i<n; i++)
      c[i][j] += a[i][k] * r;
  }
}
```

matmult/mm.c



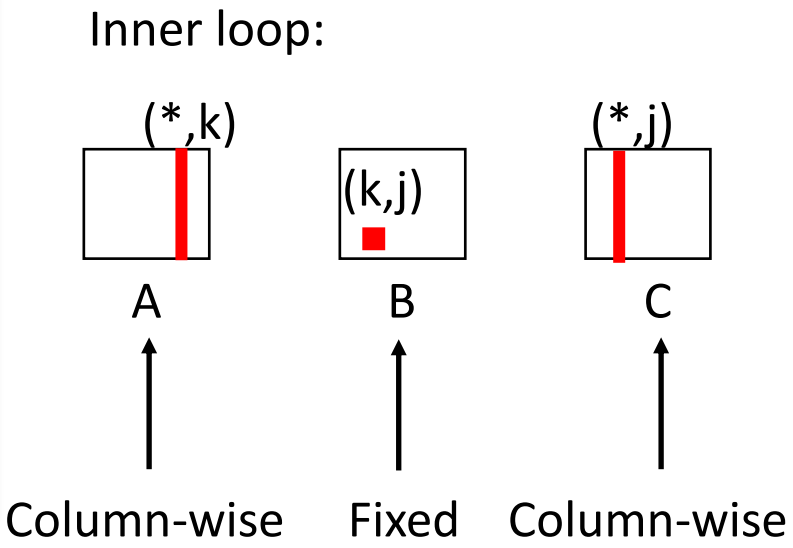

Misses per inner loop iteration:

| A | B | C |
|---|---|---|
| 1.0 | 0.0 | 1.0 |

# Matrix Multiplication (kji)

```
/* kji */
for (k=0; k<n; k++) {
  for (j=0; j<n; j++) {
    r = b[k][j];
    for (i=0; i<n; i++)
      c[i][j] += a[i][k] * r;
  }
}
```

matmult/mm.c




Misses per inner loop iteration:

| A | B | C |
|---|---|---|
| 1.0 | 0.0 | 1.0 |

# Summary of Matrix Multiplication

```
for (i=0; i<n; i++) {
  for (j=0; j<n; j++) {
   sum = 0.0;
   for (k=0; k<n; k++)
     sum += a[i][k] * b[k][j];
   c[i][j] = sum;
 }
}
```

**ijk (& jik):**

- 2 loads, 0 stores
- misses/iter = **1.25**

```
for (k=0; k<n; k++) {
 for (i=0; i<n; i++) {
  r = a[i][k];
  for (j=0; j<n; j++)
   c[i][j] += r * b[k][j];
 }
}
```

**kij (& ikj):**

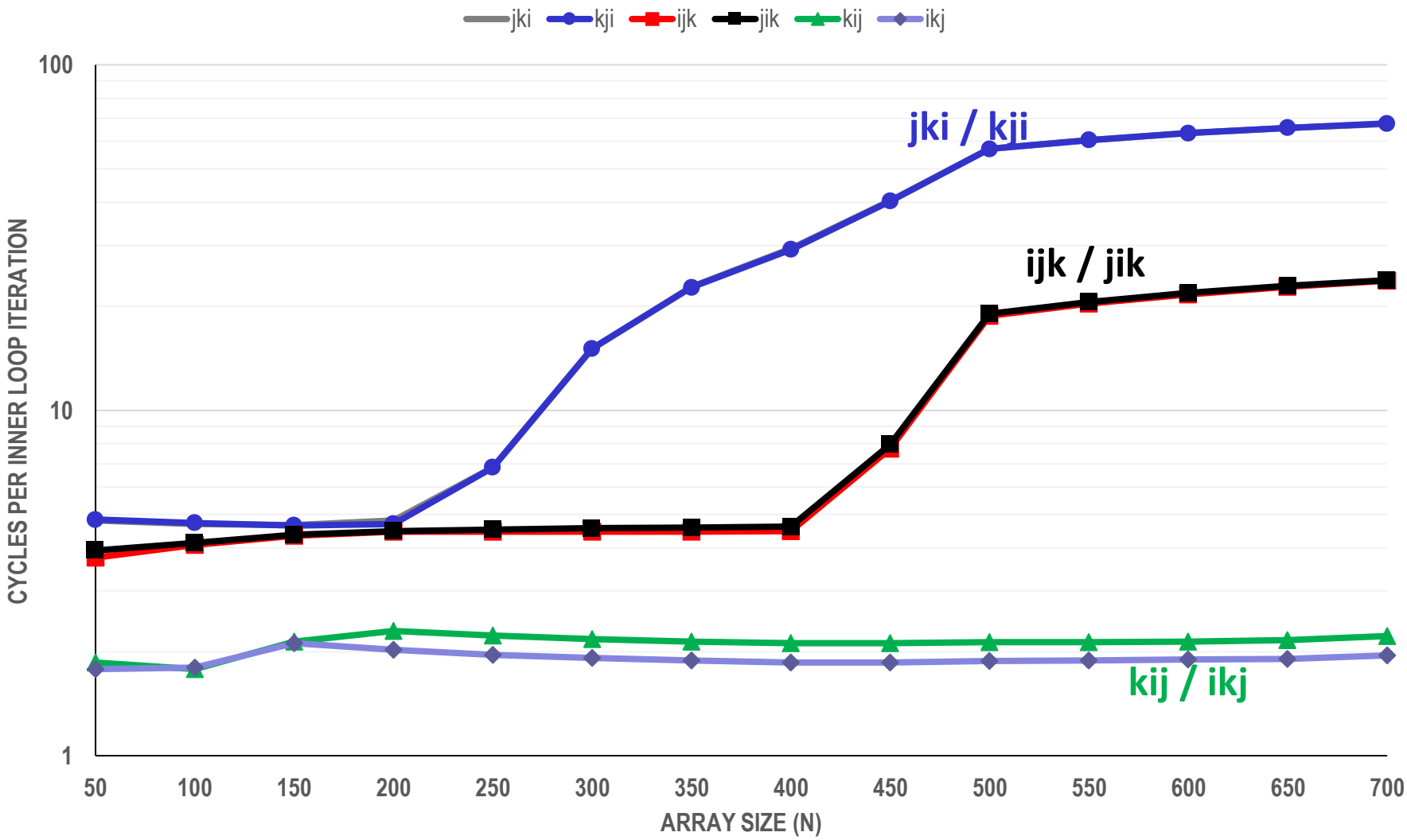
- 2 loads, 1 store
- misses/iter = **0.5**

```
for (j=0; j<n; j++) {
 for (k=0; k<n; k++) {
   r = b[k][j];
   for (i=0; i<n; i++)
    c[i][j] += a[i][k] * r;
 }
}
```

**jki (& kji):**

- 2 loads, 1 store
- misses/iter = **2.0**

# Core i7 Matrix Multiply Performance



# Today

- Cache organization and operation
- Performance impact of caches
  - Rearranging loops to improve spatial locality
  - Using blocking to improve temporal locality